

All Pair Shortest path Problem

=> it is a shortest path problem where the shortest path between every pair of vertices is computed.

=> Floyd-Warshall Algorithm

=> Johnson's Algorithm

used for solving all pairs shortest path problem.

Floyd Warshall Algorithm:

=> it is used to solve all pairs shortest path problem.

=> it computes the shortest path between every pair of vertices of the given graph.

=> It is an example of dynamic programming approach.

Pros:

=> it is extremely simple.

=> it is easy to implement

Time Complexity:

- ⇒ Floyd warshall algorithm consists of three loops over all the nodes.
- ⇒ The inner most loop consists of only constant complexity operations.
- ⇒ Hence, the asymptotic complexity of Floyd warshall algorithm is $\Theta(n^3)$.
- ⇒ Here, n is the number of nodes in the given graph.

When floyd warshall algorithm is used?

- ⇒ it is suitable for dense graphs.
- ⇒ Because its complexity depends only on the number of vertices in the graph.
- ⇒ For sparse graph, johnson's algorithm is more suitable.

Cons:

- ⇒ Complex and difficult to understand
- ⇒ can be slow down when used on large graphs
- ⇒ Not always find shortest path b/w 2-nodes

Algorithm :-

Floyd-WARSHALL(w)

$n = w.rows$

for $k = 1$ to n

Let $D^{(k)} = (d_{ij}^{(k)})$ be a new $n \times n$ matrix

for $i = 1$ to n

for $j = 1$ to n

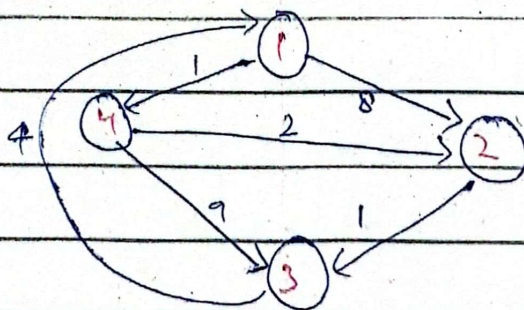
$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$$

Return D^n

Recurrence:

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k=0 \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{if } k > 1 \end{cases}$$

Shortcut method:



Step 1:

- X Remove all the Self loops and parallel edges (keeping the lowest weight edge)

Steps:

- Write the initial distance matrix.
- It represents the distance between every pair of vertices in the form of given weights.
- For diagonal elements (representing self-loops), distance value = 0.
- For vertices having direct edge b/w them, distance value = weight of that edge.
- For vertices having no direct edge between them, distance value = ∞ .

	1	2	3	4
1	0	8	∞	1
2	∞	0	1	∞
3	4	∞	0	∞
4	∞	2	9	0

	1	2	3	4
1	0	8	∞	1
2	∞	0	1	∞
3	4	12	0	5
4	∞	2	9	0

	1	2	3	4
1	0	8	9	1
2	∞	0	1	∞
3	4	12	0	5
4	∞	2	3	0

	1	2	3	4
1	0	8	9	1
2	5	0	1	6
3	4	12	0	5
4	7	2	3	0

	1	2	3	4
1	0	3	4	1
2	5	0	1	6
3	4	7	0	5
4	7	2	3	0

Applications:

- => finding shortest path
- => Detecting negative cycles
- => Computing the transitive closure of the graph.
- => Computing the diameters of the graph.

Note: It works for negative edge but not for negative cycle.

#What is Johnson's algorithm?

⇒ Solve all pairs shortest path problem in sparse weighted, directed graph.

⇒ works with the help of Dijkstra's and Bellman Ford algorithm.

⇒ For sparse graph, asymptotically faster.

⇒ Uses the technique of "reweighting."

⇒ Time complexity:

$$O(V^2 \lg V + VE)$$

⇒ if all edge weights w in graph $G = (V, E)$ are non-negative, then we find all pairs shortest path by running the Dijkstra's algorithm once for each vertex.

⇒ if G has negative weight edges, then a new set of non-negative edge weights that allows us to use the same method.

Steps: 1) Add a source node to graph, connected by zero weight edge to each other node.

2) Apply bellman ford algorithm

$$w(u,v) = w(u,v) + h(u) - h(v)$$

- if -ve cycle is detected then it will be terminated.

3) Apply Dijkstra's algorithm for finding shortest path